Sesión 18
Verificar JWT y proteger rutas

Cambios realizados y guía para tus capturas de Postman

Robert Alejandro Garcia Reyes · 27 de octubre de 2026

En la sesión 17 el servidor emitía tokens. Ahora también verifica el encabezado Authorization antes de permitir el acceso a dos rutas. El login conserva el contrato accessToken y user; el perfil protegido devuelve solo id y email.

Ruta	Política actual
POST /auth/register y POST /auth/login	Públicas: permiten obtener una cuenta y su token.
GET /auth/me	Requiere Bearer JWT válido; responde 200 con id y email.
POST /courses	Requiere el mismo JWT; conserva DTO, servicio y respuesta 201.
Demás rutas de cursos, estudiantes y matrículas	Siguen públicas en esta práctica, incluidas PATCH y DELETE de cursos.

Archivos que se añadieron o modificaron

Archivo	Responsabilidad
auth/strategies/jwt.strategy.ts	Extrae Bearer, verifica HS256 y exp, prepara identidad segura.
auth/guards/jwt-auth.guard.ts	Guard reutilizable que aplica la estrategia jwt.
auth/types/jwt-payload.type.ts	Añade AuthenticatedUser con id y email.
auth/auth.module.ts	Registra Passport, estrategia y guard; exporta el guard.
auth/auth.controller.ts	Añade GET /auth/me con @UseGuards y @Req.
courses.controller.ts y courses.module.ts	Protegen POST /courses sin cambiar su DTO ni servicio.
test y postman/sesion-18…	Prueban token inválido, efectos en base y matriz de evidencia.

El recorrido es: encabezado Bearer → JwtAuthGuard → firma y vencimiento → validate(payload) → req.user → DTO y controlador → servicio. El guard se ejecuta antes de los pipes. Un token ausente es 401; roles y permisos pertenecen a la semana 10.

1 Estrategia y guard reutilizable

Este es el código real de la estrategia. Las credenciales se verifican con el mismo JWT_SECRET de la emisión. La caducidad está activa y el algoritmo se limita explícitamente a HS256.

```
01  import { Injectable, UnauthorizedException } from '@nestjs/common';
02  import { ConfigService } from '@nestjs/config';
03  import { PassportStrategy } from '@nestjs/passport';
04  import { isEmail } from 'class-validator';
05  import { ExtractJwt, Strategy } from 'passport-jwt';
06  import type {
07    AuthenticatedUser,
08    JwtPayload,
09  } from '../types/jwt-payload.type.js';
10
11  @Injectable()
12  export class JwtStrategy extends PassportStrategy(Strategy) {
13    constructor(config: ConfigService) {
14      super({
15        jwtFromRequest: ExtractJwt.fromAuthHeaderAsBearerToken(),
16        ignoreExpiration: false,
17        algorithms: ['HS256'],
18        secretOrKey: config.getOrThrow<string>('JWT_SECRET'),
19      });
20    }
21
22    validate(payload: JwtPayload & { exp?: number }): AuthenticatedUser {
23      // Types alone do not validate JSON received inside a signed token.
24      if (
25        !payload ||
26        !Number.isSafeInteger(payload.sub) ||
27        payload.sub <= 0 ||
28        typeof payload.email !== 'string' ||
29        !isEmail(payload.email) ||
30        !Number.isSafeInteger(payload.exp)
31      ) {
32        throw new UnauthorizedException();
33      }
34
35      // Do not copy the entire payload, the database entity or its password hash.
36      return { id: payload.sub, email: payload.email };
37    }
38  }
```

El mismo guard se usa en ambos controladores

```
01  import { Injectable } from '@nestjs/common';
02  import { AuthGuard } from '@nestjs/passport';
03
04  @Injectable()
05  export class JwtAuthGuard extends AuthGuard('jwt') {}
```

validate convierte sub en id y devuelve un objeto explícito; no copia el payload completo. También valida los tipos en ejecución y exige exp. La estrategia no consulta UsersService en cada petición ni aplica reglas de rol.

Al ser una identidad tomada del token, una cuenta eliminada podría mantener acceso hasta que el token expire o cambie el secreto. Una política de estado de cuenta o revocación individual queda fuera del alcance de esta sesión.

2 Evidencia real de las rutas protegidas

La prueba registró una cuenta, hizo login y llamó a la API contra PostgreSQL temporal. El token completo se omitió en el archivo de evidencia. Esta fue la identidad del perfil autorizado:

```
GET /auth/me
Authorization: Bearer {{accessToken}}
HTTP 200

{
  "id": 1,
  "email": "sesion18-fca5b9db-2815-4ff9-a901-293000705aae@example.com"
}
```

Petición	Resultado comprobado
/auth/me con token válido	200 y solo id/email.
/auth/me sin encabezado	401.
/auth/me con firma alterada	401.
/auth/me con esquema Token	401; el esquema requerido es Bearer.
POST /courses sin token	401 y cantidad de cursos sin cambios.
POST /courses válido, nivel experto	400 por DTO y cantidad sin cambios.
POST /courses válido, nivel beginner	201 y exactamente un curso adicional.

La suite HTTP también comprobó token vencido, secreto distinto, algoritmo distinto, ausencia de exp y claims de identidad inválidos. No se llamó al servicio de creación cuando falló el guard o el DTO.

Resultados de la comprobación

Grupo	Correctas
Unitarias	31
HTTP e integración con PostgreSQL	42
Arranque e2e	1
Total de pruebas	74
Colección Postman ejecutada mediante sus scripts	13 peticiones y 32 aserciones

La ejecución automatizada de los scripts de colección se hizo contra la API y PostgreSQL reales; no fue una sesión gráfica de Postman. Tus capturas se toman con las peticiones ya preparadas.

3 Pasos para tomar las capturas en Postman

Inicia PostgreSQL y la API con npm run start:dev. Captura los nombres JWT_SECRET y JWT_EXPIRES_IN en .env.example. No muestres el .env real.

Importa postman/sesion-18.postman_collection.json y activa un entorno privado con practicePassword. La colección genera un correo único y guarda accessToken y tamperedToken en el entorno para tus peticiones manuales; no muestres ni exportes esos valores.

Captura	Petición de la colección y resultado
1 Registro	01: POST /auth/register devuelve 201.
2 Login y payload	02: abre Visualize. Muestra 200, usuario, sub/email/iat/exp y duración, sin token completo.
3 Perfil autorizado	03: GET /auth/me con Bearer {{accessToken}} devuelve 200.
4 Perfil rechazado	04 sin encabezado y 05 con {{tamperedToken}}: captura ambos 401.
5 Negocio sin token	07 conteo previo, 08 POST /courses con 401 y 09 mismo conteo.
6 Validación con token	10 nivel experto con 400 y 11 conteo sin cambios.
7 Creación con token	12 mismo curso con nivel beginner y 201; 13 total aumentado en uno.

Para capturar la expiración por HTTP, usa temporalmente JWT_EXPIRES_IN=5s en tu .env privado, reinicia la API, haz login, espera más de cinco segundos y repite /auth/me: 401. Restaura 15m, reinicia y obtén un token nuevo.

Comandos de comprobación

```
npm run build
npm run lint
npm test
npm run test:auth
npm run test:jwt
# Con PostgreSQL y variables JWT configuradas:
npm run test:jwt:guard
npm run test:integration
npm run test:e2e
```

El conteo supone que nadie más modifica cursos al mismo tiempo. No existe unicidad por título de curso: repetir un título no implica 409. El guard autentica; el DTO conserva las validaciones de título y nivel.

Guía ampliada: docs/sesion-18.md. Actividad: Sesión 18 del curso.